

docs_chain

추적된 문서는 절대 동기화에서 벗어날 수 없습니다 — 콘텐츠 해시 기반, 양방향, 원자적 처리.

요약

Docs Chain는 범용의 Go로 구현된 양방향 문서-데이터베이스 의존성 전파 엔진입니다. 등록된 체인의 구성원이 변경되면 — 마크다운 소스, HTML/PDF/DOCX 내보내기 파일, 또는 SQLite 데이터베이스 — 엔진은 콘텐츠 해시를 통해 변경을 감지하고 연결된 모든 구성원에 원자적으로 전파합니다.

간략 설명

문서와 데이터베이스를 동기화 상태로 유지하는 Go 엔진입니다. DAG(칸 위상 정렬, 조기 중단, 양방향 동기화 엣지, 원자적 이름 변경 + SQLite 트랜잭션 커밋)를 기반으로 한 셀사 스타일의 콘텐츠 해시 기반 증분 재계산을 사용하여 연결된 아티팩트가 변경될 때마다 내보내기를 재생성합니다.

상세 설명

Docs Chain는 “마크다운이 변경되면 PDF를 재생성한다”는 취약한 셀 스크립트를 한 번 더 작성한 후에야 구축하게 되는 솔루션입니다. 이 엔진은 수작업으로 작성된 동기화 코드의 전 범주를 대체합니다. 프로젝트의 문서와 데이터베이스를 체인의 구성원으로 모델링하고, 어떤 구성원이든 변경되면 선언된 모든 방향으로 그 변경을 연결된 모든 구성원에 전파하여 — 재생성 및 내보내기를 원자적으로 수행함으로써 추적된 아티팩트가 절대 동기화에서 벗어나지 않도록 합니다. 이 설계는 스크립팅이 아닌 증분 빌드 시스템의 엄격함을 그대로 차용했습니다. 변경 감지는 **mtime**이 아닌 콘텐츠 해시로 이루어지므로 `touch` 명령은 아무런 작업도 유발하지 않으며, 1바이트의 수정은 정확히 필요한 재빌드만 실행합니다 — 오탐지나 누락된 변경 없이 말입니다. 한 줄로 요약하자면, 이는 DAG를 기반으로 한 셀사 스타일의 콘텐츠 해시 기반 증분 재계산으로, 칸 위상 정렬, 조기 중단(변경되지 않은 하위 트리 가지치기), 선언된 권한을 가진 양방향 sync 엣지, 원자적 이름 변경 및 SQLite 트랜잭션 커밋을 통해 전파 중 중단되어도 불완전한 내보내기 파일이 남지 않도록 보장합니다. `vasic-digital` 서브모듈로 제공되며, `HelixConstitution` 서브모듈의 핵심 구성 요소로 사용됩니다. 따라서 현장을 채택한 모든 프로젝트는 기본적으로 Docs Chain를 활용하며, 컨텍스트

별 YAML를 통해 자체 체인을 등록합니다. 구현 상태는 헌장 §11.4.6에 따라 투명하게 공개됩니다. 1~4단계(핵심 DAG + 해싱, 노드 어댑터/변환, 원자적 전파 오케스트레이터, sync/verify/doctor/graph/watch를 지원하는 구성 기반 멀티 컨텍스트 CLI)는 구현 및 테스트 완료되었습니다. 4b단계에서는 범용 양방향 md-to-sqlite/sqlite-to-md 내장 기능(순수 Go, 행 수준 드립트, 바이트 안정적 왕복 변환)과 colorize-html 내장 기능을 추가합니다. 5단계의 종합적인 실 바이너리 e2e 테스트는 구현 완료되었으며 GREEN 상태입니다. 6~7단계(헌장 배포, ATMOSphere 연동)는 계획 단계로, 운영자 권한으로 제한됩니다. Herald는 첫 번째 실제 하류 소비자, 66개 문서의 멀티 포맷 코퍼스를 동기화하여 오류 없이 작동함을 검증합니다.

개발 배경

문서, 내보내기 파일, 데이터베이스는 수작업이나 취약한 스크립트로 관리되는 순간부터 동기화에서 벗어납니다. Docs Chain는 동기화를 기계적이며 콘텐츠 해시 기반의 정확하고 원자적인 프로세스로 전환하여, 체인의 어느 한 부분에서 변경이 발생하더라도 하류(및 상류)의 모든 요소를 안전하고 정확하게 업데이트합니다.

콘텐츠

왜 혁신적인가

컴파일러와 빌드 시스템 개발자들이 당연히 여기는, 오랜 노력 끝에 얻은 정확성 보장 기법들—콘텐츠 해시 기반 의존성 그래프, 최소 재계산, 원자적 커밋—을 이제껏 크론 잡과 선의에만 의존해왔던 문서화와 데이터베이스라는 영역에 적용합니다. 진정한 양방향 동기화는 소스와 그 출력물 간의 관계를 양방향으로 강제하므로, “문서가 최신 상태가 아니다”나 “출력물이 소스와 일치하지 않는다”와 같은 문제는 더 이상 반복되는 버그가 아니라 엔진이 허용하지 않는 상태가 됩니다.

혁신적인 점

- 콘텐츠 해시(수정 시각 아님)를 기반으로 한 DAG 상의 증분 재계산 및 조기 중단.
- 양방향, 명시적 권한 동기화 간선(문서 ↔ 출력물 ↔ SQLite).
- 원자적 이름 변경 + SQLite 트랜잭션 커밋을 통한 충돌 안전 전파.
- 순수 Go 기반 md-to-sqlite/sqlite-to-md 왕복 변환 및 행 수준 변화 감지.

도전 과제와 해결 방안

- 불필요한 재빌드: 콘텐츠 해시 감지로 해결(타임스탬프 대신).
- 부분적/손상된 업데이트: 원자적 이름 변경과 SQLite 트랜잭션으로 해결.

- 다중 멤버 정렬의 정확성: 칸 위상 정렬 + 조기 중단으로 해결.
- 정직한 기능 보고: 각 단계별 구현 상태(\$11.4.6 기준 IMPLEMENTED vs PLANNED)를 명시하여 해결.

기술 스택(이유와 방법)

- **Go** — 전체 엔진(internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain).
- **DAG + 칸 위상 정렬** — 조기 중단 기능이 있는 의존성 정렬.
- **SQLite(순수 Go modernc)** — DB 멤버 및 트랜잭션 커밋.
- **fsnotify** — 실시간 전파를 위한 watch 데몬.
- **YAML** 설정 — 컨텍스트별 체인 등록.
- **exec: 변환** — 플러그형 Markdown→HTML/PDF/DOCX 생성.

로드맵 투명성: 6~7단계(헌법 배포, ATMOSphere 연동)는 PLANNED / 운영자 제한 상태—아직 출시되지 않음.